

# FPCP3

```
library(tidyverse)
library(tidymodels)
library(readxl)
# Load data
df = read.csv("GrowthDJ.csv")
head(df)
nrow(df)
ncol(df)

# df after cleaning
df_clean <- na.omit(df)
head(df_clean)
nrow(df_clean)
ncol(df_clean)

class(df_clean$oil)
df_clean$oil <- as.factor(df_clean$oil)
```

## Cross Validation (CV)

1. Write a brief introduction to cross validation which includes relevant mathematical notation.

Answer: Cross validation is a statistical method used in machine learning to evaluate how well a model performs in an unseen or independent data set. It involves dividing the data set into multiple folds or subsets, using one of these folds as a validation set and training the model on the remaining folds. This process is then repeated multiple times each time using different fold as the validation set. The main purpose of cross validation is to prevent over fitting which occurs when a model is trained too well on the training data and performs poorly on new, unseen data

$$CV(k) = \frac{1}{k} \sum_{j=1}^k MAE_j$$

2. What is the goal of cross-validation? (Hint: Think about over-fitting, along with social/ethical considerations)

Answer: The goal of cross validation is to ensure that a model generalizes well beyond the data on which it was trained by repeatedly holding out different subsets for testing, it guards against over fitting and produces an approximately unbiased estimate of out of sample error. This process not only helps in selecting and tuning models that truly capture underlying patterns rather than noise, but also promotes robustness and fairness: by validating across different folds.

3. What linear models are you considering based on your research question? Pick at least two models to compare.

Answer: I am considering two models. One model which is unadjusted model (model\_1) which doesn't contain confounders and another model which is adjusted model (model\_2) that includes the confounders.

```
# Step 1: Model specification
lm_spec <- linear_reg() %>%
# Output Y is quantitative
set_mode("regression") %>%
# Want regression to be linear
set_engine("lm")
```

```
# Unadjusted model
model_1 <- lm_spec %>% fit(gdpgrowth ~ oil, data = df_clean)
```

```
# Adjusted model
model_2 <- lm_spec %>% fit(gdpgrowth ~ oil + school + invest, data = df_clean)
```

4. Explain how you divided your data into its test set and training set.

Answer: I split the data into two parts –a training set (80% of the observations) and a test set (the remaining 20%) so that we could both tune our models and then evaluate their performance on truly unseen data.

```
set.seed(2025)
split      <- initial_split(df_clean, prop = 0.8, strata = "oil")
train_data <- training(split)
test_data  <- testing(split)
```

5. State which error metric you are using (MAE or MSE) and give its formal mathematical definition. Why did you choose this error metric? What are the advantages/disadvantages of using it?

Answer: I will be using MAE. The formal mathematical definition is:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

I chose this error metric because it produces an error metric in the same units as the outcome which makes it immediately interpretable and it is also less sensitive to large outliers than the other error metric like MSE. The main advantage of MAE is the robustness to outliers. Since errors aren't squared a few extreme misses don't dominate the metric. Another advantage is that it's easy to compute and understand which doesn't require additional transformations.

However, it does have some drawbacks like MAE treats every error the same so one huge miss and several small misses can look equally bad even though we might just care about the big one.

6. Implement k-fold cross validation for  $k = 10$ .

Answer:

```
# 10-fold cross validation for model_1
set.seed(244)
model_1_cv <- lm_spec %>%
  fit_resamples(gdpgrowth~oil,
                resamples = vfold_cv(train_data, v = 10),
                metrics = metric_set(mae, rsq)
  )
```

```
# 10-fold cross validation for model_2
set.seed(253)
model_2_cv <- lm_spec %>%
  fit_resamples(
    gdpgrowth~oil+school+invest,
    resamples = vfold_cv(train_data, v = 10),
    metrics = metric_set(mae, rsq)
  )
```

7. Display evaluation metrics for your different models in a clean, organized way. This display should include both the estimated CV metric as well as its standard deviation.

Answer:

```
model_1_cv %>%
  collect_metrics()
```

```
model_2_cv %>%
  collect_metrics()
```

```
# In sample MAE for model_1
model_1 %>%
  augment(new_data = train_data) %>%
  mae(truth = gdpgrowth, estimate = .pred)
```

```
# In sample MAE for model_2
model_2 %>%
  augment(new_data = train_data) %>%
  mae(truth = gdpgrowth, estimate = .pred )
```

```
model_1_cv %>% unnest(.metrics) %>%
  filter(.metric == "mae") %>%
  summarize(sd(.estimate))
```

```
model_2_cv %>% unnest(.metrics) %>%
  filter(.metric == "mae") %>%
  summarize(sd(.estimate))
```

| Model   | IN-SAMPLE MAE | 10-fold CV MAE | Standard Deviations |
|---------|---------------|----------------|---------------------|
| model_1 | 1.50          | 1.54           | 0.337               |
| model_2 | 1.46          | 1.51           | 0.340               |

8. Try different values of k (the tuning parameter). At minimum, try  $k = n - 1$  (LOOCV), and  $k = 5$ . Which value of k has the smallest CV error?

Answer: 5-fold CV gave the lowest mean MAE, so we will use  $k=5$ .

```
# LOOCV
model_1_loocv <- lm_spec %>%
  fit_resamples(gdpgrowth~oil,
    resamples = vfold_cv(train_data, v = nrow(train_data)),
    metrics = metric_set(mae)
  )
```

```
model_1_loocv %>%
  collect_metrics()
```

```
# 5-fold cv
set.seed(244)
model_1_cv5 <- lm_spec %>%
```

```
fit_resamples(
  gdpgrowth~oil,
  resamples = vfold_cv(train_data, v = 5),
  metrics = metric_set(mae, rsq)
)
```

```
model_1_cv5 %>%
  collect_metrics()
```

```
model_2_loocv <- lm_spec %>%
  fit_resamples(
    gdpgrowth~oil+school+invest,
    resamples = vfold_cv(train_data, v = nrow(train_data)),
    metrics = metric_set(mae)
  )
```

```
model_2_loocv %>% collect_metrics()
```

```
# 5-fold cv
set.seed(253)
model_2_cv5 <- lm_spec %>%
  fit_resamples(
    gdpgrowth~oil+school+invest,
    resamples = vfold_cv(train_data, v = 5),
    metrics = metric_set(mae, rsq)
  )
```

```
model_2_cv5 %>%
  collect_metrics()
```

9. Select your final model based on which one has the smallest CV error.

Answer: Model 2 has the smallest CV error compared to model 1. So, model\_2 is my final model.

## LASSO

1. Write a brief introduction to LASSO which includes relevant mathematical notation.

Answer: LASSO is a penalized regression method that shrinks coefficient estimates and performs variable selection.

$$RSS + \lambda \sum_{j=1}^p |\beta_j|$$

where  $\lambda \geq 0$  is a tuning parameter controlling the strength of the L1 penalty. As  $\lambda$  grows more coefficients are shrinking towards 0 or exactly to 0 which yields a model that is easier to interpret and often stable in the presence of multicollinearity. LASSO's key advantages are its built in variable selection and improved predictive performance via regularization and its main drawback is potential bias in the non-zero coefficients due to shrinkage.

2. State which error metric you are using (i.e., CV MAE or CV MSE and state the chosen k-value).

Answer: I will be using CV MAE and k=10.

3. Identify at least  $p = 3$  predictors  $X_1, X_2, \dots, X_p$  for modeling the expected response  $E(Y)$  of one of your variables  $Y$ .

Answer: For the LASSO model, I will predict

$Y = \text{gdpgrowth}$

Using at least  $p=3$  predictors.

$X_1 = \text{gdp60}$

$X_2 = \text{invest}$

$X_3 = \text{school}$

4. Compute 50 LASSO models for 50 different values of  $\lambda$  (the penalty tuning parameter), each with all  $p$  predictors. You can start out by trying  $\lambda$  in the range of -5 to 1 on the log10 scale. Plot the cross-validated (CV) error for each value of  $\lambda$ .

Answer:

```
# STEP 1: LASSO Algorithm & Model Specification
# This is copied exactly from the R Code Notes for LASSO Section
lasso_spec <- linear_reg() %>%
  set_mode("regression") %>%
  set_engine("glmnet") %>%
  set_args(mixture = 1, penalty = tune())
```

```
# STEP 2: Variable Recipe
variable_recipe <- recipe(gdpgrowth ~ gdp60 + invest + school, data = train_data) %>%
  step_dummy(all_nominal_predictors())
```

```
# STEP 3: Workflow Specification (Model + Recipe)
# This is copied exactly from the R Code Notes for LASSO Section
lasso_workflow <- workflow() %>%
  add_recipe(variable_recipe) %>%
  add_model(lasso_spec)
```

```
# STEP 4: Estimate 50 LASSO models using
# lambda values on a "grid" or range from 10^(-5) to 10^(-0.1).
# Calculate the 10-fold CV MAE for each of the 50 models.
set.seed(123)
lasso_models<-lasso_workflow%>%
  tune_grid(
    grid= grid_regular(penalty(range=c(-5,1)),levels= 50 ),
    resamples= vfold_cv(train_data, v= 10 ),
    metrics= metric_set(mae)
  )
```

```
autoplot(lasso_models)+
  scale_x_continuous() +
  xlab(expression(lambda))
```

5. Based on your plot in the previous step, what value of lambda is best?

Answer: Based on my plot, when  $\lambda = 0.34$

```
# Checking the approximation with the code
best_penalty <- lasso_models %>%
  select_best(metric = "mae")
best_penalty
```

6. Pick the lambda associated with the parsimonious model.

Answer: 10

```
# This is copied exactly from the R Code Notes for LASSO Section
autoplot(lasso_models) +
  scale_x_continuous() +
  xlab(expression(lambda)) +
  geom_errorbar(data = collect_metrics(lasso_models),
    aes(x = penalty, ymin = mean- std_err, ymax = mean +
      std_err), alpha = 0.5
  )
```

```
parsimonious_penalty <- lasso_models %>%  
  select_by_one_std_err(metric = "mae", desc(penalty))  
parsimonious_penalty
```

7. For your parsimonious model, how many and which predictors were kept?

Answer: For my parsimonious model, 4 variables were kept but rest of the other variables were shrunk exactly to zero. The variables that were kept are: gdp60, popgrowth, invest and inter\_yes

```
final_lasso <- lasso_workflow %>%  
  finalize_workflow(parameters = parsimonious_penalty) %>%  
  fit(data = train_data)  
final_lasso %>%  
  tidy()
```